Testbench Generation Assignment Report

Course: LLM4ChipDesign

Instructors: Prof. Ramesh Karri

Overview

This report describes the implementation of an LLM-aided workflow for automatic Verilog testbench generation and verification. The goal of the assignment was to use the provided notebook to generate testbenches, construct a reference (golden) model, and validate hardware designs using simulation.

The pipeline uses an LLM to generate both stimulus and expected outputs, and then integrates them into a self-checking testbench. This reduces the need for manual testbench writing while still ensuring correctness through simulation.

The workflow was applied to:

- Part I: Two tutorial examples (MUX and 4-bit adder)
- Part II: Inspection of generated artifacts
- Part III: Custom module (priority encoder)
- Part IV: Bug detection and correction

All designs were verified using iverilog and vvp.

System Architecture

The pipeline follows a step-by-step process:

1. **Input specification**
   A natural-language description of the DUT behavior is provided along with the Verilog module. The quality of this description directly affects the correctness of later stages.
2. **Initial testbench generation**
   The LLM generates testbench_initial.v, which includes input stimulus and a set of test patterns. At this stage, the testbench only drives inputs and does not check correctness.
3. **Golden model generation**
   A Python file (golden_model.py) is generated based on the specification. This acts as a reference implementation that computes expected outputs for given inputs.

4. **Pattern generation with expected outputs**
   The golden model is executed on all generated inputs. The results are stored in test_patterns_with_golden.json, which contains both inputs and expected outputs.
5. **Testbench update**
   The initial testbench is modified to produce testbench_final.v. This version includes comparison logic, pass/fail counters, and summary reporting.
6. **Simulation**
   The design is compiled using iverilog and simulated using vvp. The DUT outputs are compared against the expected outputs from the JSON file.

This structure separates stimulus generation, reference behavior, and verification logic, making the pipeline easier to debug and extend.

Model and Configuration

- Base URL: https://integrate.api.nvidia.com/v1
- Model: meta/llama-3.1-8b-instruct
- API: OpenAI-compatible

The model was used for:

- Generating testbenches
- Writing the golden model
- Producing structured test patterns

The results depend on how clearly the behavior is described in the prompt. Ambiguous specifications can lead to incorrect golden models.

Key Features

- Automatic generation of Verilog testbenches
- Python-based golden reference model
- JSON storage of test inputs and expected outputs
- Self-checking testbench with pass/fail reporting
- End-to-end verification using simulation tools

Part I - Tutorial Examples

Example 1: 2-to-1 Multiplexer

Behavior:

- Output equals a when sel = 0
- Output equals b when sel = 1

This example validates basic combinational behavior and ensures that the pipeline correctly handles simple logic.

Result:

- Summary: 8 passed, 0 failed

Example 2: 4-bit Adder

Behavior:

- Computes the sum of two 4-bit inputs
- Produces carry-out when overflow occurs

This example tests arithmetic correctness and multi-bit operations.

Result:

- Summary: 4 passed, 0 failed

Part II - Artifact Inspection

Golden Model (golden_model.py)

The golden model defines the expected behavior of the DUT in Python. For example, in the MUX case:

- Inputs: a, b, sel
- Output: y = a if sel == 0 else b

This model is used to compute expected outputs independently of the DUT implementation.

Pattern File (test_patterns_with_golden.json)

Each entry in the JSON file includes:

- Input values
- Expected outputs computed from the golden model

This allows the testbench to directly compare DUT outputs with correct values during simulation.

Updater Effect

Initial testbench:

- Applies input patterns
- Does not check correctness

Final testbench:

- Loads expected outputs
- Compares DUT outputs against them
- Tracks passed and failed tests
- Prints a final summary

This step converts the testbench into a self-checking verification environment.

Part III - Custom Module

Module: 4-bit Priority Encoder

Functionality:

- Input: d[3:0]
- Output:
    - valid: indicates if any bit is 1
    - enc[1:0]: index of the highest set bit

Priority is given to the most significant bit.

Test Coverage

The pipeline generated around 20 test cases, including:

- All-zero input
- Single-bit cases
- Multiple-bit cases
- Random combinations

This ensures that both normal and edge cases are covered.

Result

- Summary: 20 passed, 0 failed

Part IV - Bug Detection

Bug Introduction

A bug was introduced in the DUT (incorrect priority handling).

Result:

- Summary: 7 passed, 13 failed

The mismatches confirm that the testbench correctly detects functional errors.

Bug Fix

After correcting the DUT:

Result:

- Summary: 20 passed, 0 failed

Final Verification Commands

- iverilog -g2012 -o mux mux2to1.v mux/testbench_final.v
  vvp mux
- iverilog -g2012 -o adder adder4bit.v adder/testbench_final.v
  vvp adder
- iverilog -g2012 -o custom priority_encoder4.v custom/testbench_final.v
  vvp custom
- iverilog -g2012 -o custom_buggy priority_encoder4_buggy.v
  custom/testbench_final.v
  vvp custom_buggy

Conclusion

The LLM-based testbench generation pipeline successfully automates both stimulus creation and verification for Verilog designs.

Key observations:

- Golden models provide a reliable way to validate functionality
- Separating test patterns and expected outputs improves reproducibility
- Self-checking testbenches simplify debugging
- The approach works for both simple and moderately complex designs

Overall, the workflow shows that LLMs can be effectively used as a tool in hardware verification, as long as results are validated through simulation.